

Software Design Document (SDD)

Project: Smart University Bus Info System (SUBIS)

Version: 1.0

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Scope	1
1.3 Definitions and Acronyms	1
1.4 References	1
2. System Overview	2
2.1 System Environment	2
2.2 Design Constraints	2
2.3 Technology Stack	2
3. System Architecture	3
3.1 Architectural Design (MERN Stack)	3
3.2 Decomposition Description	3
3.3 Communication Interfaces (WebSocket/REST)	3
4. Data Design	4
4.1 Database Schema (MongoDB)	4
4.2 Data Dictionary	4
5. Component Design	5
5.1 User Authentication Module	5
5.2 Tracking & Location Module	5
5.3 Route & Stop Management Module	5
5.4 Crowd Estimation Module	5
6. User Interface Design	6
6.1 Student Interface	6
6.2 Admin Dashboard	6
7. Requirements Matrix (Traceability from SRS to SDD)	7

1. Introduction

1.1 Purpose

The purpose of this Software Design Document (SDD) is to describe the architectural and detailed design of the **Smart University Bus Information System (SUBIS)**. This document traces the requirements defined in the SRS to specific software modules and data structures. It is intended for the development team to use as the primary blueprint for implementing the system using the MERN stack.

1.2 Scope

SUBIS is a real-time transportation tracking system designed for university students and administrators.

- **The System Will:**

- Track bus locations in real-time using GPS data.
- Calculate and display Estimated Time of Arrival (ETA) for stops.
- Provide a web interface for students to view routes and crowd levels.
- Provide an admin dashboard for managing routes, stops, and bus assignments.
- **The System Will Not:**
 - Handle fare collection or payments.
 - Manage driver payroll or HR functions.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
SUBIS	Smart University Bus Info System
MERN	MongoDB, Express.js, React.js, Node.js (The chosen tech stack)
JWT	JSON Web Token (Used for secure authentication)
Socket.io	Library for real-time, bi-directional communication (used for live tracking)
OSM	OpenStreetMap (The mapping provider)
ETA	Estimated Time of Arrival

2. System Overview

2.1 System Environment

SUBIS operates as a web-based application accessible via modern web browsers (Chrome, Firefox, Safari) on both desktop and mobile devices. It relies on a client-server model where the client (React.js) communicates with the server (Node.js/Express) via HTTPS and Secure WebSockets (WSS).

2.2 Design Constraints

- **Real-Time Latency:** Location updates must be reflected on the user map within 1-3 seconds.
- **Map Licensing:** The system must use OpenStreetMap (OSM) to avoid Google Maps licensing fees.

- **Connectivity:** The system must handle intermittent network connections from mobile devices gracefully.
- **Privacy:** Student location data used for crowd estimation must be anonymized before storage.

2.3 Technology Stack

- **Frontend:** React.js (with Leaflet.js for maps)
- **Backend:** Node.js with Express.js
- **Database:** MongoDB (NoSQL)
- **Real-time Engine:** Socket.io
- **Authentication:** University Email (SMTP) + JWT

3. System Architecture

3.1 Architectural Design

The **Smart University Bus Information System (SUBIS)** follows a **Client-Server Architecture** utilizing the **MERN Stack** (MongoDB, Express.js, React.js, Node.js). This architecture is chosen for its efficiency in handling JSON data across all layers and Node.js's ability to handle concurrent real-time connections (WebSockets) essential for live bus tracking.

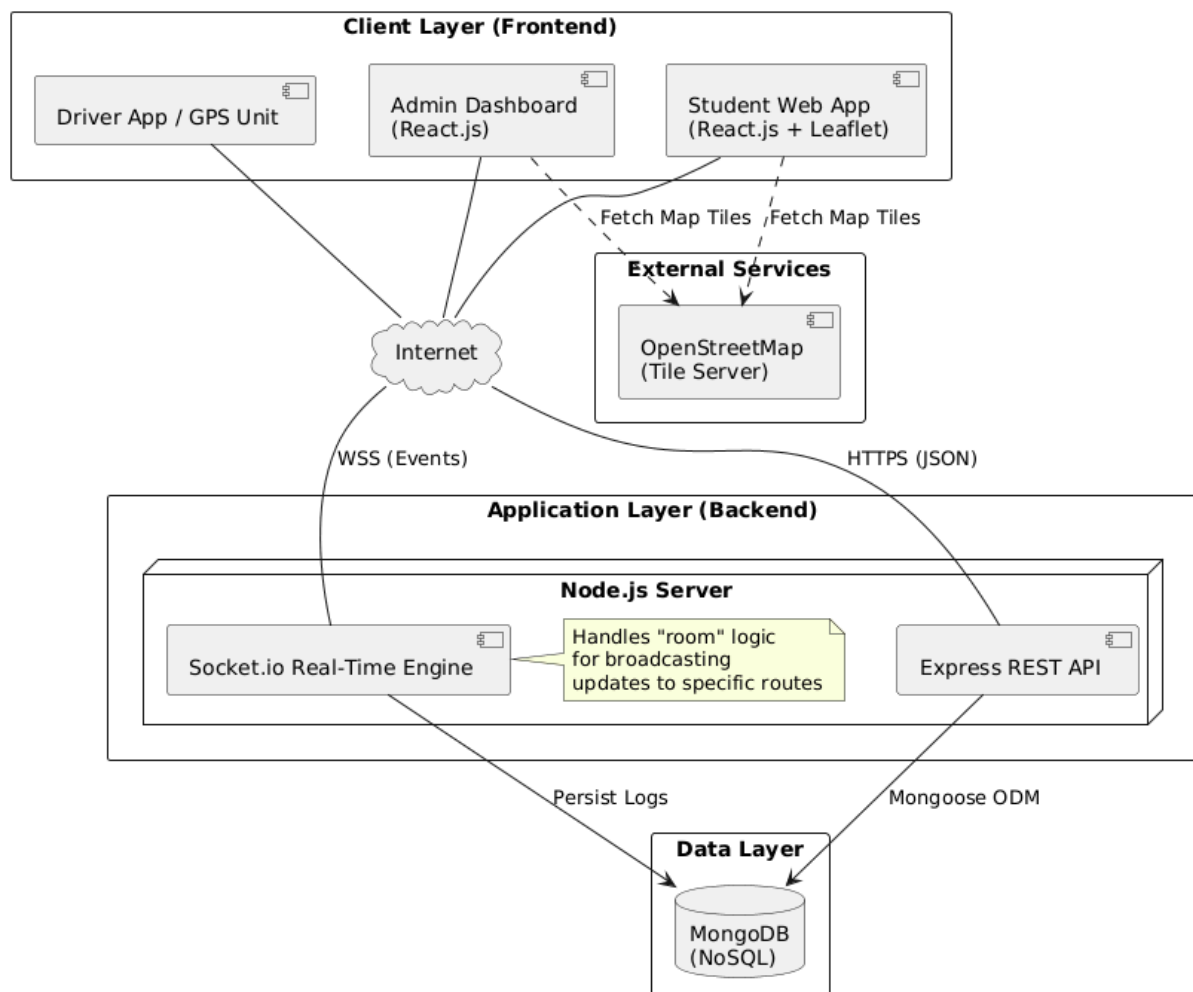


Figure 3.1 - System Architecture The system follows a client-server architecture using the MERN stack.

- **Clients:** The React-based interfaces interact with the server via **REST API** for authentication and route management, and via **Secure WebSockets (WSS)** for live bus tracking.
- **Server:** The Node.js server hosts both the Express API and the Socket.io engine, serving as the central hub for data processing.
- **External:** The frontend directly fetches map tiles from **OpenStreetMap** to reduce server load.

The system is divided into three main layers:

1. **Presentation Layer (Client):**
 - **React.js Application:** The user interface for students and administrators. It manages the application state and renders dynamic maps using **Leaflet.js**.
 - **Communication:** Connects to the backend via **REST APIs** (for static data like routes/user profiles) and **Socket.io Client** (for live location updates).
2. **Application Layer (Server):**
 - **Node.js & Express.js:** The core backend server. It handles authentication, business logic (ETA calculation, crowd estimation), and API routing.
 - **Socket.io Server:** A dedicated module within the Node.js server that listens for GPS telemetry streams and broadcasts updates to connected clients.
3. **Data Layer (Database):**
 - **MongoDB:** A NoSQL database used to store persistent data (Users, Routes, Logs). It stores location data in **GeoJSON** format for efficient geospatial querying.

3.2 Decomposition Description

The system is decomposed into the following functional modules:

- **User Authentication Module:**
 - **Responsibility:** Manages registration, login, and password resets using university email.
 - **Tech:** Uses **JSON Web Tokens (JWT)** for session management and **SMTP** for email verification .
- **Real-Time Tracking Engine:**
 - **Responsibility:** Receives GPS telemetry from buses/students and broadcasts it to active users.
 - **Tech:** **Socket.io**. It handles "rooms" (e.g., a room for "Route A") so users only receive relevant updates .
- **Route & Stop Management Module:**
 - **Responsibility:** Allows Admins to create, edit, and delete bus routes and stops.
 - **Tech:** REST API endpoints (GET, POST, PUT, DELETE) interacting with the **Routes** collection in MongoDB .

- **Crowd Estimation Module:**
 - **Responsibility:** Aggregates location data from student devices (who have opted-in) near a specific bus or stop to calculate density levels (Low/Moderate/High) .

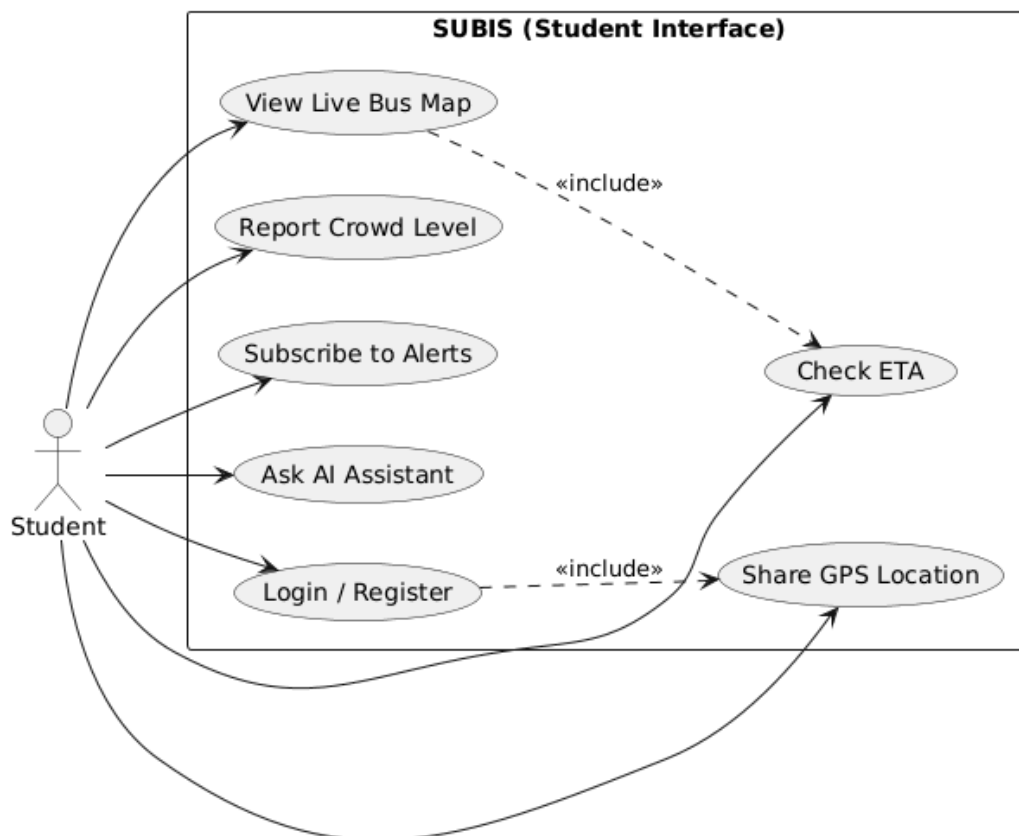
3.3 Communication Interfaces

- **REST API:** Used for non-real-time actions.
 - Format: JSON.
 - Protocol: HTTPS.
 - Example: `GET /api/v1/routes`, `POST /api/v1/auth/login`.
- **WebSocket:** Used for real-time bi-directional events.
 - Protocol: WSS (Secure WebSocket).
 - Events: `bus_location_update`, `eta_change`.

3.4 Use Case Diagrams

This section describes the primary interactions between the actors (users) and the system, highlighting the functional requirements from a user's perspective.

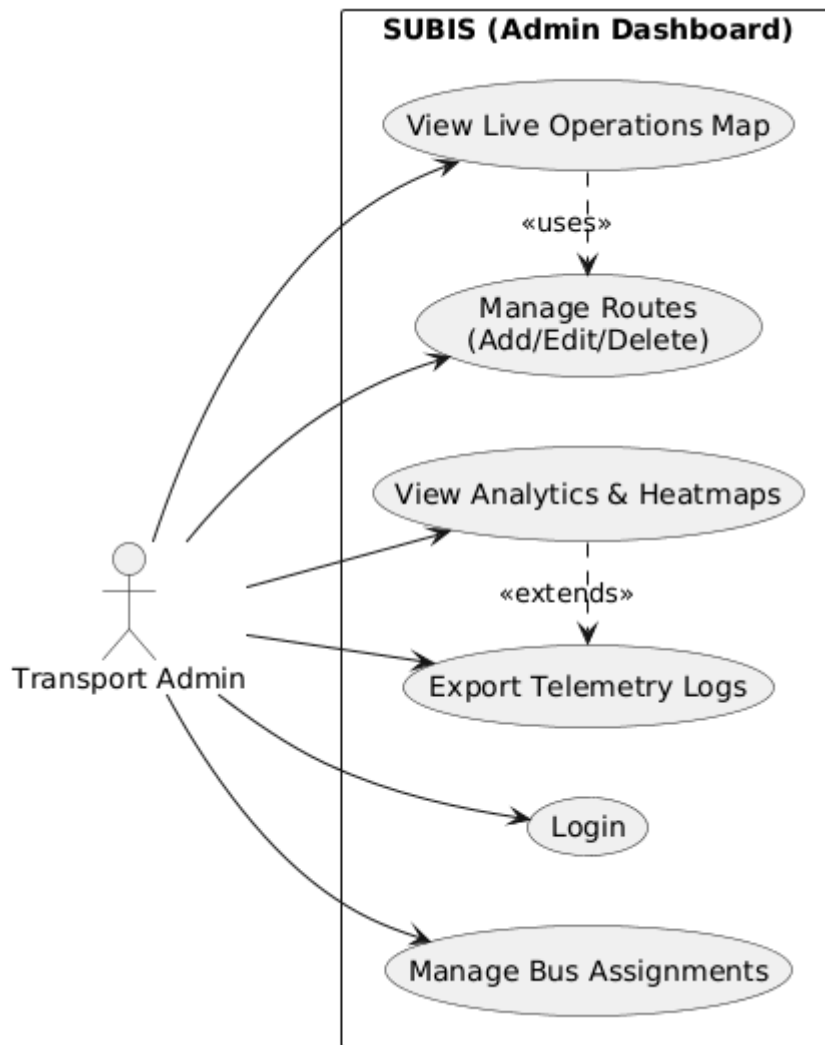
Figure 3.2 - Student Use Case Diagram



Description: The Student view focuses on real-time information retrieval.

- **View Live Bus Map:** The central use case, which includes checking ETAs for specific stops.
- **Share GPS Location:** This is an "include" relationship with Login; students must authenticate to opt-in for crowd estimation data sharing.
- **Report Crowd Level:** Allows students to manually contribute data regarding bus capacity.

Figure 3.3 - Admin Use Case Diagram



Description: The Admin view focuses on system management and oversight.

- **Manage Routes:** Involves the CRUD (Create, Read, Update, Delete) operations for the static route data.
- **View Analytics:** Extends the functionality of the dashboard to show historical performance and crowd heatmaps.
- **Manage Bus Assignments:** Allows the admin to link a specific bus (by plate number) to a specific route.

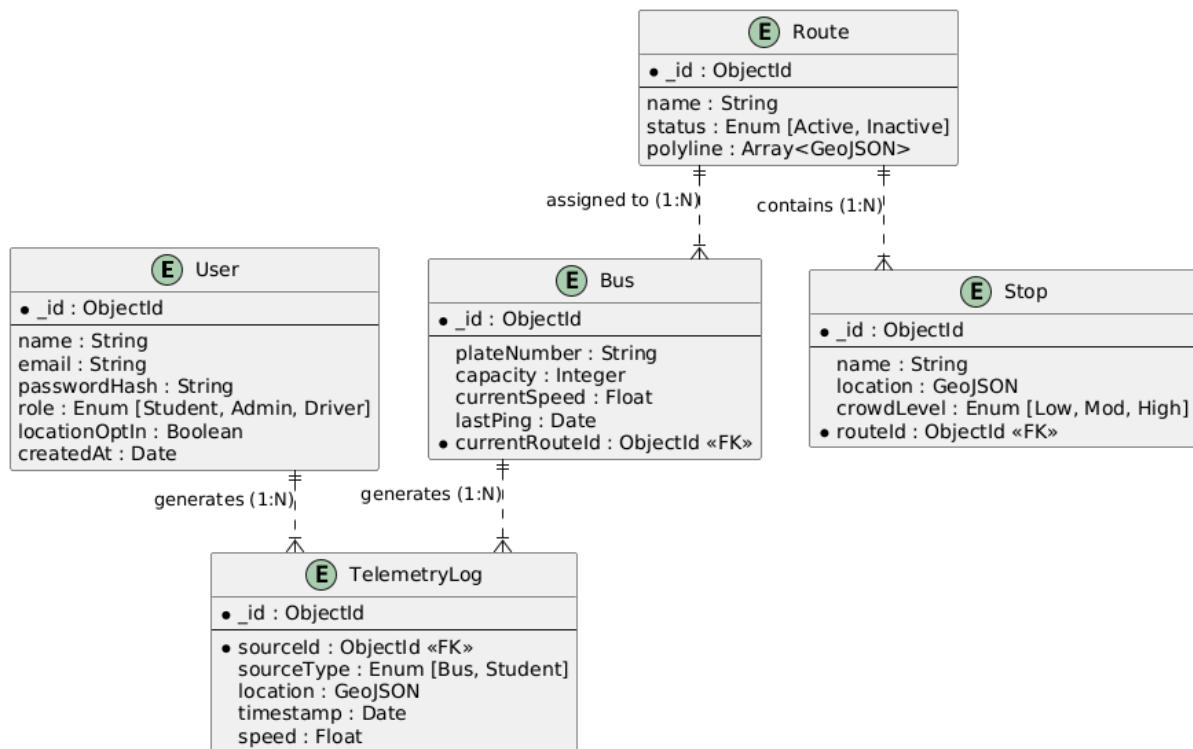
4. Data Design

4.1 Data Description

SUBIS uses **MongoDB**, a document-oriented database. Data is stored in collections of JSON-like documents. This allows for flexible schemas, which is particularly useful for nesting stops within routes or storing varying telemetry data.

4.2 Data Dictionary (Database Schema)

Below are the schema definitions corresponding to the entities defined in the SRS:



1. Users Collection (**users**)

Stores student and admin account information.

Field	Type	Description
Field	Type	Description
<code>_id</code>	ObjectId	Unique identifier

		(Auto-generated).
name	String ▾	Full name of the user.
email	String ▾	University email (Unique, Indexed).
passwordHash	String ▾	Encrypted password string.
role	String ▾	Enum: ['student', 'admin', 'driver'].
locationOptIn	Boolean ▾	True if user agrees to share location data.
createdAt	Date ▾	Account creation timestamp.

2. Routes Collection (routes)

Stores the static path and stops for a bus line.

Field	Type	Description
_id	ObjectId ▾	Unique identifier.
routeNumber	String ▾	e.g., "Route 12", "Campus Loop A".
stops	Array ▾	Array of objects: [{ stopId, sequenceOrder, eta }].
polyline	Array ▾	List of [lat, lng] points drawing the path on the map.
status	String ▾	Enum: ['active', 'inactive'].

3. Stops Collection (stops)

Stores specific bus stop locations.

Field	Type	Description
_id	ObjectId ▾	Unique identifier.
name	String ▾	e.g., "Main Gate", "Library", "Science Block".
location	GeoJSON ▾	{ type: "Point", coordinates: [lng, lat] }.

crowdLevel	String ▾	Enum: ['low', 'moderate', 'high'] (Calculated).
------------	----------	---

4. Buses Collection (buses)

Stores the current state of a vehicle.

Field	Type	Description
_id	ObjectId ▾	Unique identifier.
plateNumber	String ▾	Vehicle registration number.
currentRoute	ObjectId ▾	Reference to routes collection.
location	GeoJSON ▾	Current { lat, lng } of the bus.
speed	Number ▾	Current speed in km/h.
lastUpdate	Date ▾	Timestamp of the last GPS ping.

5. Telemetry Logs (telemetry)

Stores historical data for analytics (speed, location history).

Field	Type	Description
_id	ObjectId ▾	Unique identifier.
sourceId	ObjectId ▾	Reference to buses or users collection.
type	String ▾	Enum: ['bus', 'student'].
location	GeoJSON ▾	Snapshot of location.
timestamp	Date ▾	Time of recording.

5. Component Design

This section details the internal logic of the system's key modules. It defines the Application Programming Interfaces (APIs) and the logic flow for critical processes defined in the SRS¹¹¹¹.

5.1 Authentication Module (User Controller)

Responsible for managing student and admin access via university email²²²².

- **Component ID:** `AuthController`
- **Responsibilities:** Login, Registration, JWT Token Generation, Email Verification.

- **API Endpoints:**

Method	Endpoint	Input Parameters	Output	Description
POST	/api/v1/auth/register	name, email, password, role	201 Created	Hashes password (bcrypt), creates user, sends OTP email.
POST	/api/v1/auth/verify	email, otp	200 OK	Verifies email ownership; activates account.
POST	/api/v1/auth/login	email, password	token (JWT)	Validates credentials; returns session token.

5.2 Real-Time Tracking Module (Socket Service)

This module handles the bi-directional communication for live location updates using **Socket.io**. It corresponds to FR-03 (Bus Tracking) and FR-04 (ETA)³³³³.

- **Component ID:** TrackingService
- **Logic Flow:**
 1. **Listen:** Listens for **update_location** events from Bus Clients (GPS trackers/Driver Phones).
 2. **Process:**
 - Validate coordinates.
 - Update the **buses** collection in MongoDB with new **location** and **lastUpdate** timestamp.
 - *Trigger:* Recalculate ETA for next stops based on new distance.
 3. **Broadcast:** Emits **bus_moved** event to all Student Clients subscribed to that specific route room (e.g., **room_route_12**).

5.3 Route Management Module (Admin)

Allows administrators to manage the static data of the transport network⁴.

- **Component ID:** RouteController
- **API Endpoints:**

Method	Endpoint	Input Parameters	Output	Description

GET	/api/v1/routes	None	Array[Route]	Fetches all active routes and their stops.
POST	/api/v1/routes	name, stops[], polyline	Route Object	Creates a new bus route.
PUT	/api/v1/routes/:id	status ('active'/'inactive')	200 OK	Toggles route availability.

5.4 Crowd Estimation Module

Implements FR-06 by aggregating data to determine density⁵.

- **Component ID:** CrowdService

Algorithm (Pseudo-code):

JavaScript

FUNCTION CalculateCrowdLevel(stopId):

// 1. Get count of active users with GPS enabled near stop location

```
userCount = DB.Users.count({
  location: { $near: { $geometry: stopCoordinates, $maxDistance: 50m } },
  lastActive: { $gt: 5_minutes_ago }
})
```

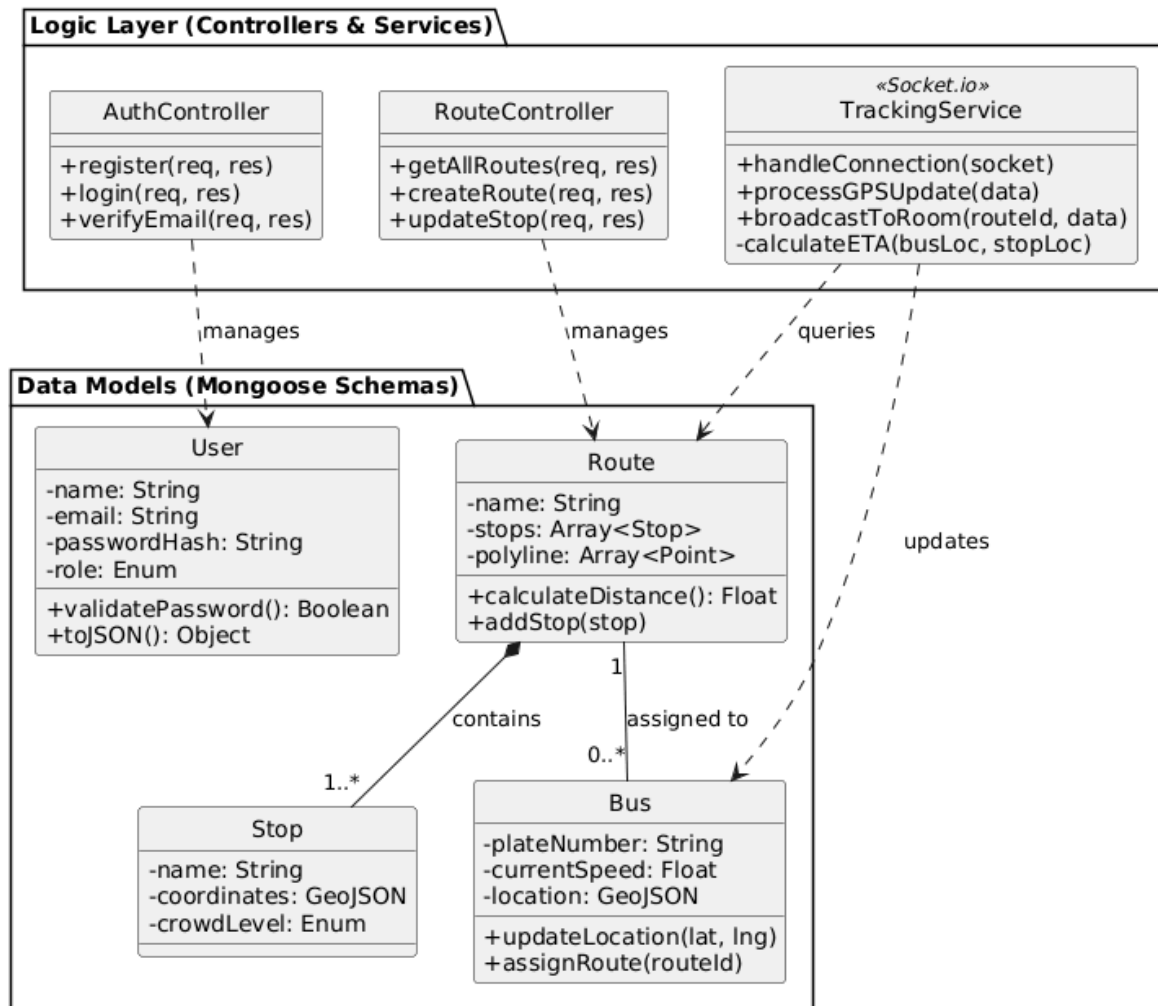
// 2. Determine Level

IF userCount < 10 THEN RETURN "Low"

ELSE IF userCount < 30 THEN RETURN "Moderate"

ELSE RETURN "High"

5.5 Class Diagrams



5.6 Sequence Diagrams

Live Bus Tracking:

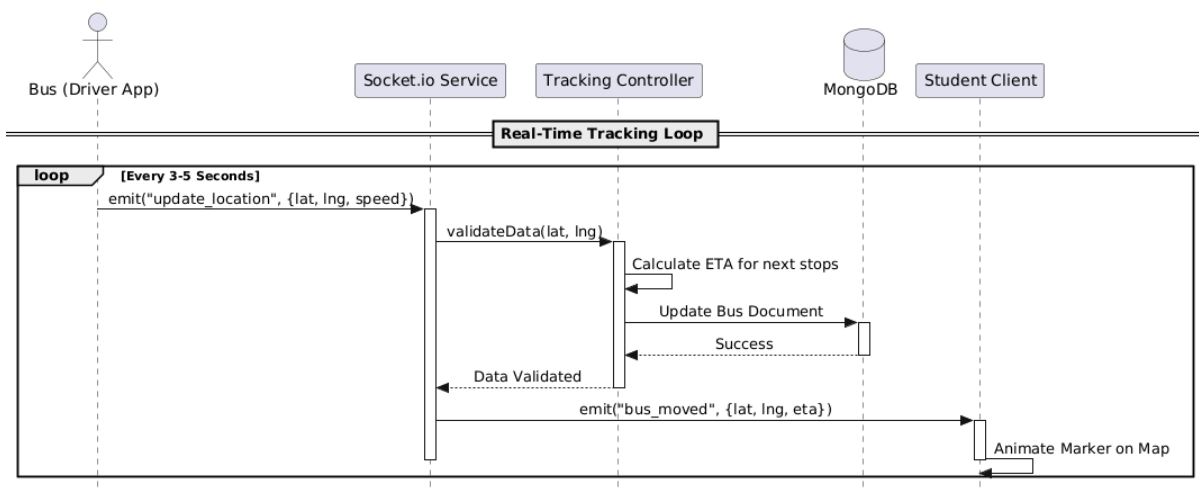


Figure 5.2 - Live Bus Tracking Sequence This illustrates the bi-directional communication flow. The Bus sends telemetry to the server, which processes it and

immediately broadcasts the update to subscribed students without requiring a page refresh.

User Login:

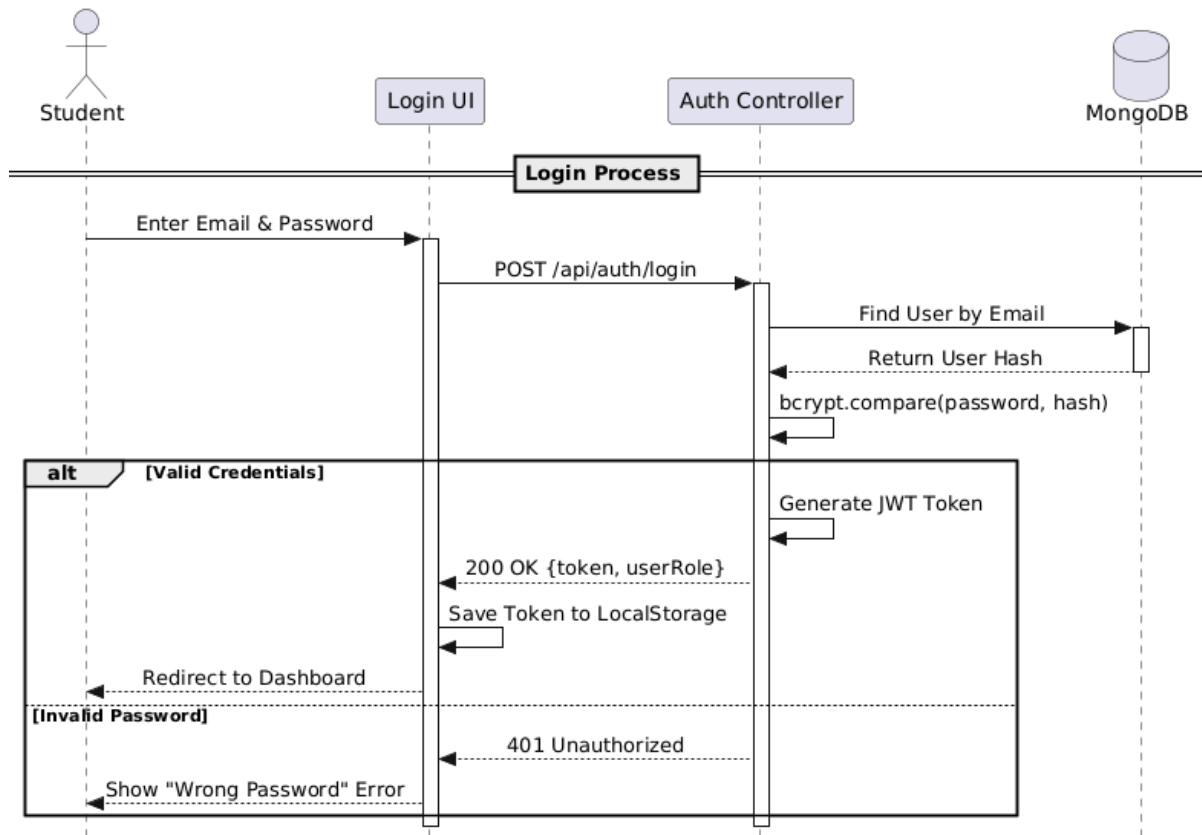


Figure 5.3 - User Authentication Sequence This details the secure login flow. It demonstrates the interaction between the client and the Auth Controller, including password verification (using bcrypt) and session token generation (JWT).

Add Route:

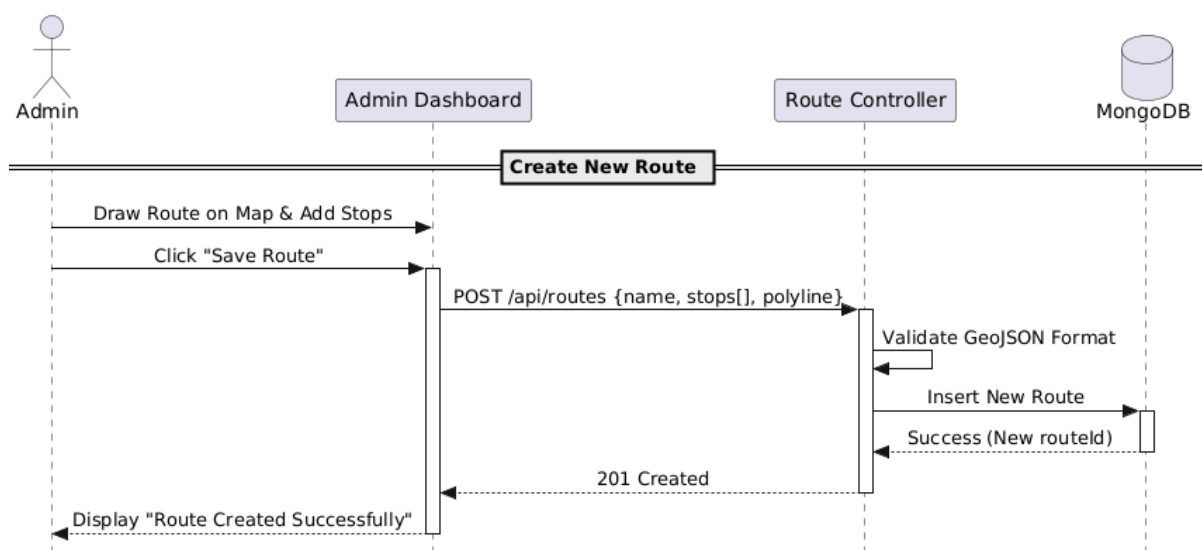


Figure 5.4 - Admin Route Creation Sequence This illustrates how administrators interact with the system to define static network data. It highlights the validation and storage of geospatial data in MongoDB.

6. User Interface Design

This section describes the "Look and Feel" and user flows for the React.js frontend. It aligns with the user characteristics defined in the SRS⁶.

6.1 Student Interface (Mobile/Web)

Designed for speed and ease of access while on the go.

- **Screen 1: The Live Map (Home)**
 - **Layout:** Full-screen interactive map (Leaflet.js).
 - **Overlays:** Moving bus icons (color-coded by route), Bus Stop markers.
 - **Interactions:** Tapping a bus icon opens a "Bottom Sheet" displaying:
 - Bus Number / Plate.
 - Next Stop Name.
 - **ETA (Large Font)**⁷.
 - Crowd Level Indicator (Green/Yellow/Red icons).
- **Screen 2: Route Selection**
 - **Layout:** A searchable list of all university routes (e.g., "Blue Line", "Campus Loop").
 - **Action:** Clicking a route highlights its path on the Home Map and filters out other buses.
- **Screen 3: Notifications & Alerts**
 - **Layout:** List of system broadcasts (e.g., "Bus 12 delayed due to traffic").
 - **Controls:** Toggle switches for "Notify me when bus is 5 mins away"⁸.

6.2 Admin Dashboard (Desktop)

Designed for management and analytics⁹.

- **Screen 1: Operations Overview**
 - **Visuals:** A "God-view" map showing all active buses simultaneously.
 - **Sidebar:** List of active alerts (e.g., "Bus 04 not responding", "Route B High Traffic").
 - **Screen 2: Route Manager**
 - **Layout:** Split screen. Left side lists routes; Right side shows a map editor.
 - **Action:** Admins can click and drag markers on the map to adjust bus stop locations or redraw route polylines.
 - **Screen 3: Analytics**
 - **Visuals:**
 - Bar Chart: Peak usage hours (Students vs. Time).
 - Heatmap: Most crowded stops on campus.
-

7. Requirements Matrix

SRS Req ID	Requirement Name	Handling Module (SDD Section)	Implementation Component (Class/Service)
FR-01	User Authentication	5.1 User Auth ... ▾	AuthController, UserSchema
FR-02	User Profile Management	5.1 User Auth ... ▾	AuthController (Update Profile)
FR-03	Real-Time Bus Tracking	5.2 Tracking ... ▾	TrackingService, Socket.io Engine
FR-04	ETA Calculation	5.2 Tracking ... ▾	TrackingService (Algorithm)
FR-05	Route Management	5.3 Route Mo... ▾	RouteController, RouteSchema
FR-06	Crowd Estimation	5.4 Crowd Mo... ▾	CrowdService
FR-07	Bus Stop Management	5.3 Route Mo... ▾	RouteController
FR-08	Notification System	5.2 Tracking ... ▾	Socket.io (Event Emitter)
FR-09	Admin Dashboard	6.2 Admin UI ▾	AdminDashboard.jsx (React)
FR-10	Student Map Interface	6.1 Student UI ▾	LiveMap.jsx (React/Leaflet)
FR-11	Location Privacy (Opt-in)	4.2 Data Design ▾	UserSchema.locationOptIn (Boolean)